

System Design Exercise - EM

System Design Interview Exercise — Engineering Manager (Software Development)

Duration: 60 minutes **Role Level:** Engineering Manager

Problem Statement

Design a Real-Time Order Management System for a Multi-Restaurant Food Delivery Platform

A fast-growing food delivery company is launching in 10 new cities next quarter. You are the Engineering Manager responsible for designing and leading the build of the **Order Management System (OMS)** — the core service that handles the lifecycle of an order from placement to delivery completion.

The system must support:

1. Customers placing orders from restaurant menus
2. Restaurants receiving and accepting/rejecting orders
3. Real-time order status tracking (placed, confirmed, preparing, out-for-delivery, delivered)
4. Delivery partner assignment and location tracking
5. Notifications to all parties (customer, restaurant, delivery partner) at each status change

Current scale (Day 1): - 50,000 daily active users - 5,000 orders per day - 500 restaurants

Target scale (12 months): - 5 million daily active users - 500,000 orders per day - 50,000 restaurants across 15 cities

What the Candidate is Expected to Deliver

1. Assumptions (documented explicitly)

The candidate should state and justify assumptions including but not limited to:

- Peak traffic patterns (e.g., lunch/dinner spikes — 3-5x average load)
- Average order lifecycle duration
- Acceptable latency thresholds for different operations
- Consistency vs. availability trade-offs for different parts of the system
- Third-party dependencies (payment gateway, mapping/routing service)
- Mobile-first or web-first user base

What good looks like: Assumptions are realistic, explicitly stated, and directly inform design decisions. The candidate proactively identifies ambiguity and resolves it before designing.

2. High-Level Technical Design

The candidate should produce a system architecture diagram (whiteboard/drawing) that includes:

- **Client Layer:** Mobile apps, restaurant dashboard, delivery partner app
- **API Gateway / Load Balancer**
- **Core Services:** Order Service, Restaurant Service, Delivery Service, Notification Service, User Service
- **Async Communication:** Message queue / event bus (e.g., Kafka, SQS) for decoupling
- **Data Stores:** Primary database(s), cache layer, search index if applicable
- **Real-Time Layer:** WebSocket or SSE for live order tracking
- **External Integrations:** Payment provider, maps/routing API, SMS/push notification provider

What good looks like: Services are appropriately bounded. Synchronous vs. asynchronous communication is deliberately chosen. The candidate can articulate *why* they separated (or combined) services, not just *what* they drew.

3. API Contract

The candidate should define key API endpoints with request/response structure.

What good looks like: APIs are versioned, use proper HTTP methods and status codes, have clear naming, and the candidate discusses idempotency, authentication (JWT/OAuth), and rate limiting.

4. Data Model

The candidate should define the core entities and their relationships:

What good looks like: The candidate snapshots prices at order time (not referencing current menu prices), uses an audit/history table for status changes, and can justify their database choice with trade-offs.

5. Non-Functional Requirements

The candidate must address each of the following:

a) Latency

Operation	Target	Approach
Place order	< 500ms (p99)	Async payment capture, write-through cache
Status update propagation	< 2s to all parties	Event-driven via Kafka + WebSocket push
Order history query	< 200ms	Read replicas, caching hot orders
Delivery partner location update	< 1s	Redis geospatial, in-memory processing

b) Scalability

- **Horizontal scaling** of stateless services behind a load balancer
- **Database sharding strategy:** Shard by `city_id` or `restaurant_id` for geographic locality
- **Read replicas** for query-heavy operations (order history, restaurant dashboards)
- **Event-driven architecture** to decouple services — Order Service publishes events; Notification, Analytics, and Billing services consume independently
- **Auto-scaling policies** based on orders-per-second metrics during peak hours

c) Handling Large User Base (100x growth)

- **CDN** for static assets (menus, images)

- **Rate limiting** per user and per restaurant to prevent abuse
- **Circuit breakers** on external dependencies (payment, maps)
- **Graceful degradation:** If notification service is down, orders still process; notifications retry via dead-letter queue
- **Database connection pooling** (PgBouncer or equivalent)
- **Eventual consistency** for non-critical reads (e.g., estimated delivery time can be a few seconds stale)

d) Reliability & Fault Tolerance

- **Idempotent order creation** (idempotency key in request header) to prevent duplicate charges
- **Saga pattern** or **choreography** for distributed order lifecycle (order placement -> payment -> restaurant confirmation)
- **Dead-letter queues** for failed event processing with alerting
- **Multi-AZ deployment** for database and services
- **Health checks and readiness probes** for all services

e) Observability

- Structured logging with correlation IDs (trace an order across all services)
- Metrics: orders/sec, p50/p95/p99 latency, error rates, payment success rate
- Distributed tracing (OpenTelemetry / Jaeger)
- Alerting on SLO breaches (e.g., order placement > 1s p99 triggers page)

What good looks like: The candidate prioritizes trade-offs (e.g., “I’d accept eventual consistency on delivery ETA but require strong consistency on payment status”). They tie NFRs back to specific architectural decisions rather than listing buzzwords.
